DEPI | Digital Egypt Pioneers Initiative

Tour Guide App

Project Documentation: Parts 1 to 3

Planning & Management | Literature Review | Requirements Gathering

Program	DEF · Digital Egypt Pioneers Initiative (Phase 5)
Track	Mobile Development (Flutter / Dart)
Project name	Tour Guide App
Team leader	M1 -
Team	5 members (M1 to M5, see Section 1.3)
[redacted]	[redacted]
[redacted]	[redacted]
Version / date	v2.0 - 19 October 2026

Contents: Part 1 Project Planning & Management | Part 2 Literature Review | Part 3 Requirements Gathering

Part 1

Project Planning & Management

1.1 Project Proposal

Overview. Tour Guide App is a Flutter mobile application that helps tourists, and Egyptians visiting another city, discover attractions. The user signs up, tells the app where they live, which city they are visiting, where they are staying and what they like (history, nature, beaches, food, shopping). The app then shows **featured places that match those interests, closest first, with the distance**. The user can open a place for details (photos, description, opening hours, rating), open directions in Google Maps, save places into a personal visit list and trips, and get an alert when they are close to a saved place.
Problem. Information about attractions is spread over many apps and websites; visitors do not know which places are near them or match their taste; and they have no single place to keep the list of places they want to see and to be reminded when they are close to one.

Objectives

- **O1.** List historical and recreational attractions from an external places service, with search and filters.
- **O2.** Personalize the app with the user's residence, visited city, hotel and interests, and show the nearest places first with distance.
- **O3.** Support navigation by opening directions in Google Maps with one tap.
- **O4.** Alert the user when they are near a place saved in their visit list.
- **O5.** Let the user keep a visit list (add, remove, mark visited, notes) and group places into trips.
- **O6.** Deliver a tested app and complete documentation on time for every DEPI deadline, with an equal share of work for all 5 members.

1.1.1 Scope

In scope	Out of scope (future work)
Login and sign up (email or Google); profile with residence, visited city, hotel and interests; settings (alerts on/off, language) Attractions list (historical / recreational) with search, filters, featured places and a details page Distance to each place, nearest-first sorting, open in Google Maps Nearby alerts for saved places; visit list and trips 2 or 3 bonus features per the team plan (ratings, offline mode, share trip, badges, day planner)	Booking tickets, hotels or payments Turn-by-turn navigation inside the app (we hand off to Google Maps) AI chatbot, translation and image recognition Admin dashboard for managing places Web or desktop versions

Note: the original project brief mentions displaying locations on a map with navigation. Our plan covers this by opening each place and its directions in Google Maps; an embedded in-app map can be added later if the instructor requires it.

1.1.2 Technology Stack (to be confirmed in the Design step)

Layer	Choice	Owner
Mobile app	Flutter / Dart (Android, iOS if a device is available)	All
Login and database	Firebase Authentication (email, Google) and Cloud Firestore (free plan)	M4 sets up, M1 and M2 use
Places data (external API)	One places service chosen and tested by M2 (candidates: OpenTripMap or Google Places)	M2
Distance	Device location + distance calculation between user (or hotel) and place	M3
Directions	Open Google Maps through a link / intent with the place coordinates	M3
Location and alerts	Location permission, periodic location reading, local notifications	M4
Collaboration	GitHub (one repository), Figma for screens, team chat	M1 (leader), M3 (design)

1.2 Project Plan

Roadmap. The team plan has 7 steps. We assigned dates to them so that they fit the official DEPI deadlines. Steps overlap a little: M1 (login) and M4 (Firebase) start first because everyone needs them.

#	Step	Dates (2026)	Goal
1	Plan	until 19 Oct	Agree on features, write requirements, literature review, plan and test plan; upload to GitHub
2	Design	20 Oct - 6 Nov	Draw all screens, plan the database, set up Firebase, choose the places service. **DEPI deadline: System Analysis & Design (6 Nov)**
3	Build 1	31 Oct - 12 Nov	Basics: login, city, attractions list, details page, distance
4	Build 2	13 - 20 Nov	Google Maps buttons, visit list and trips, nearby alerts, settings. **Main app works**
5	Extras	21 - 25 Nov	One bonus feature per member and polish
6	Test	24 - 30 Nov	Test on real phones and fix bugs. **DEPI deadline: Implementation (30 Nov)**
7	Finish	28 Nov - 4 Dec	Thesis, slides, demo video, final app file. **DEPI deadline: Final Presentation, Testing & Reports (4 Dec)**; then practice for the defense

1.2.1 Gantt Chart

Step	19-25 Oct	26 Oct-1 Nov	2-8 Nov	9-15 Nov	16-22 Nov	23-29 Nov	30 Nov-4 Dec
1. Plan							
2. Design							
3. Build 1							
4. Build 2							
5. Extras							
6. Test							
7. Finish							

Bars show the weeks each step is active (approximate); the exact dates are in the table above.

1.2.2 Team Timeline (who does what in each step)

	Plan	Design	Build 1	Build 2	Extras	Test	Finish
Team	Plan together					Test (all)	Thesis + defense
M1	Requirements		Login	Onboarding + settings	**Ratings**	Tests M2	Thesis book
M2	Places service	Database	Attractions	Featured + details	**Offline mode**	Tests M3	
M3		Draw screens	Distance	Maps buttons	**Share trip**	Tests M4	
M4		Firebase setup	Location	Nearby alerts	**Badges**	Tests M5	Final app
M5	Test plan		Visit list	Trips	**Day planner**	Leads testing	

Colored cell = main work (member color); outlined cell = bonus feature; grey cell = side job. M1 login and M4 Firebase come first because everyone needs them; M3 and M5 wait for M2's attractions list; M4's nearby alerts wait for M5's visit list.

1.2.3 Milestones

#	Milestone	Date	Exit criteria
M1	Documentation parts 1-3 submitted	19 Oct	One PDF uploaded to the team GitHub repository; project name and link registered; LMS task completed by the team leader
M2	Design delivered	6 Nov	All screens drawn, database plan, Firebase project ready, places service tested; System Analysis & Design documents on GitHub
M3	Basics working (Build 1)	12 Nov	Login, attractions list, details page and distance work on a phone
M4	Main app works (Build 2)	20 Nov	All 9 must-have features work together: maps buttons, visit list and trips, nearby alerts, settings
M5	Extras done	25 Nov	Each member's bonus feature is finished, or cut and documented
M6	Implementation delivered	30 Nov	Tested on real phones; source code, README and app file on GitHub
M7	Final delivery	4 Dec	Test report, user manual, technical documentation, presentation, infographic, thesis and demo video

1.2.4 Deliverables

DEPI step	Deliverables
1. Planning, 2. Literature Review, 3. Requirements (19 Oct)	This document: proposal, plan, roles, risks, KPIs, review of similar apps, stakeholders, user stories, requirements
4. System Analysis & Design (6 Nov)	Problem statement, use case diagram, architecture, database design and ERD, DFD, sequence, activity, state and class diagrams, screens and UI/UX guidelines, tech stack, API documentation, test plan
5. Implementation (30 Nov)	Source code with comments and naming conventions, GitHub history, README (installation, requirements, run guide), packaged app file
6. Testing & 7. Final (4 Dec)	Test cases and bug reports, user manual, technical documentation, presentation, infographic, demo video, thesis

1.2.5 Resource Allocation

Resource	Allocation
People	5 members; each one has the same structure: 1 main part, 1 bonus feature, 1 side job, testing a teammate's part, a thesis chapter, and a part of the defense
Tools	Flutter SDK, VS Code / Android Studio, GitHub (one repository), Figma, Firebase (free plan), a places API free tier
Devices	At least 2 real Android phones for location and alert testing (the Test step); emulators for the rest
Budget	Target: zero cost. Google Maps is opened by link, so no paid map SDK is needed
Process	Weekly meeting, short task board, pull requests reviewed by another member, shared Definition of Done

1.3 Task Assignment & Roles

Each member builds **one main part, one bonus feature and has one side job**. Everyone also tests a teammate's part, writes about their own part in the thesis and presents it at the defense. Replace the placeholders with real names.

Member	Main part	Bonus feature	Side job	Tests part of	Name
M1	Login, profile, residence, city, likes, settings	Ratings and reviews	Team leader	M2	[Name]
M2	Attractions list and details page	Offline mode	Database	M3	[Name]
M3	Distance and Google Maps	Share trip	App design	M4	[Name]
M4	Nearby alerts	Badges	Publishing the app (and Firebase setup)	M5	[Name]
M5	Visit list and trips	Day planner	Tester (leads testing)	M1	[Name]

1.3.1 Step-by-Step Guide for Each Member

Step	M1: Login, city and settings (team leader)	Type
Plan	Write what the login and onboarding screens should do	Side job
Build 1	Build the login and sign-up screens (email or Google)	Main
Build 2	Build 'choose your residence' and 'choose where you are visiting'; the hotel and 'what do you like' screens; settings (alerts on/off, language)	Main
Extras	Ratings and reviews	Bonus
Test	Test M2's part, then write the thesis part	Side job
Finish	Put the thesis book together	Side job

Step	M2: Attractions (database)	Type
Plan	Pick the places service and test it	Main
Design	Plan how data is saved in the database	Side job
Build 1	Get the places and show them in a list	Main
Build 2	Add search and the Historical / Recreational filters; show featured places based on likes; build the place details page	Main
Extras	Offline mode	Bonus
Test	Test M3's part, then write the thesis part	Side job

Step	M3: Distance and maps (app design)	Type
Design	Draw all the app screens	Side job
Build 1	Calculate the distance from the user to each place; sort the list so the nearest places come first	Main
Build 2	Add the 'Open in Google Maps' and 'Navigate' buttons	Main
Extras	Share a trip	Bonus
Test	Test M4's part, then write the thesis part	Side job

Step	M4: Nearby alerts (publishing)	Type
Design	Set up the Firebase project for the team	Side job
Build 1	Ask the user for location permission; read the user's location every few minutes	Main
Build 2	Show an alert when the user is close to a saved place	Main
Extras	Badges	Bonus
Test	Test M5's part, then write the thesis part	Side job
Finish	Build the final app file	Side job

Step	M5: Visit list (tester)	Type
Plan	Write the test plan	Side job
Build 1	Build 'add to list' and 'remove from list'; add 'mark as visited' and notes	Main
Build 2	Group places into trips	Main
Extras	Day planner	Bonus
Test	Test M1's part, lead the team testing, write the thesis part	Side job

Dependencies: M3 and M5 start their main work after M2's attractions list exists; M4's nearby alerts need M5's visit list. Until then they use sample data.

1.3.2 Testing Rotation and Working Agreements

- Testing chain: M1 tests M2, M2 tests M3, M3 tests M4, M4 tests M5, M5 tests M1 and leads the team testing.
- One GitHub repository; branches *main* (stable), *develop* and *feature/<name>*; no direct pushes to main; pull requests are reviewed by another member.
- Weekly team meeting led by M1; a blocked member tells the team within 24 hours so tasks can be shared instead of one person carrying extra work.
- A task is done when it works, is reviewed, is merged, and its documentation is on GitHub.

1.4 Risk Assessment & Mitigation Plan

Likelihood (L) and Impact (I) are rated 1 (low) to 3 (high); Score = L x I. Each member owns 3 risks. Red = 6-9, yellow = 4-5, green = 1-3.

ID	Risk	L	I	Score	Mitigation / contingency	Owner
R1	Google sign-in or Firebase Auth setup problems (keys, consent screen)	2	3	6	Follow the official setup guide early (Design step); test on a real phone; keep email login as a fallback	M1
R2	Personal data exposed (profile, residence, hotel, location)	1	3	3	Collect only what is needed; Firestore rules so each user reads and writes only their own data; keys out of GitHub	M1
R3	Compressed timeline and members' exams or other courses	3	3	9	Parallel work, weekly check, equal task structure; cut bonus features first; leader rebalances tasks	M1
R4	Places API limits, cost or changed terms	2	3	6	Test the service in the Plan step; cache results in the database; keep a second API as backup	M2
R5	Missing or wrong place data (photos, hours, rating)	2	2	4	Show defaults when data is missing; filter bad records; store cleaned data in Firestore	M2
R6	Late database changes break other members' work	2	2	4	Agree on the data structure in the Design step; change it only through the team; use a shared data model	M2
R7	Distance is wrong when location is off or user denies permission	2	2	4	Fall back to the hotel or city center; show a clear message; allow manual city choice	M3
R8	Google Maps link behaves differently on devices (app not installed, iOS)	2	2	4	Use a standard maps URL with a browser fallback; test on at least 2 phones	M3
R9	Screen designs arrive late and block others	2	2	4	Draw the main screens first (login, list, details); share Figma early; others start with simple layouts	M3
R10	Phone limits background location or alerts (battery saver, permissions)	3	3	9	Test on real phones early; explain permissions to the user; read location every few minutes only; document limits	M4
R11	Battery drain or repeated alerts annoy the user	2	2	4	Long check interval, one alert per place per period, alerts on/off in settings	M4
R12	Firebase limits or app file signing and publishing problems	2	2	4	Stay within the free plan; build and sign a test app file before the Test step	M4
R13	M5 and M4 wait for M2's list (dependency chain) and fall behind	3	2	6	Use sample data from day 1; M2 delivers the list first in Build 1; weekly dependency check	M5
R14	Bugs found too late because testing is near the end	2	3	6	M5 writes the test plan in Plan step; everyone tests small parts during Build 1 and 2 as well	M5
R15	Too many bonus features (scope creep)	2	2	4	Bonus only after the 9 must-have features work; cut or postpone a bonus if a deadline is at risk	M5

1.5 KPIs (Key Performance Indicators)

Measured in the Test step (24-30 Nov) on real phones. "Uptime" means that core features stay usable, using saved data when the internet or the places service is down.

Category	KPI	Target	How measured
Performance	Attraction list load time	< 3 s on mobile data; < 1 s from saved data	Stopwatch / Flutter DevTools
Performance	App start time	< 3 s on a mid-range Android phone	Flutter DevTools
Performance	Distance calculation and sorting for 100 places	< 1 s	Unit test timing
Reliability	Crash-free sessions	99% or higher	Firebase Crashlytics
Reliability	Feature availability (uptime)	99% of core flows usable, with saved-data fallback	Test scenarios incl. offline
Accuracy	Nearby alert delay	Alert within 5 minutes after entering the radius	Walking test with a real phone
Accuracy	Duplicate or false alerts	< 5% of test visits	Test log
Efficiency	Battery use with alerts on	< 5% per hour	Phone battery statistics
Quality	Automated test coverage	60% or higher for business logic	flutter test --coverage
Quality	Open critical / high bugs at delivery	0	Bug reports
Quality	Test reports	5 of 5 members tested a teammate's part	Test reports on GitHub
Usability	System Usability Scale (SUS)	70 or higher	Survey of 15 testers
Usability	First-time setup (login to featured places)	Finished in < 2 minutes by 90% of testers	Observed user tests
Adoption	Beta testers	At least 20 (students, friends, family)	Install and login counts
Adoption	Return use	40% of testers open the app on 2 or more different days	Analytics events
Adoption	Saved places per tester	3 or more on average	Visit list data
Project	DEPI deadlines met	4 of 4 (19 Oct, 6 Nov, 30 Nov, 4 Dec)	Submission dates
Project	Must-have features finished	9 of 9	Feature checklist
Project	Bonus features finished	At least 3 of 5	Feature checklist
Team	Equal contribution	Every member finished main part, bonus or side jobs, testing and thesis part	Task board and Git history

Part 2

Literature Review

2.1 Purpose and Method

We looked for existing apps and projects that solve a similar problem: finding attractions, saving places, planning trips, and being guided by location. We reviewed six solutions (three international, three Egypt-related), using their public App Store pages, websites and news articles in October 2026, listed in the references. We did not install every app, so each feature below is what the source describes; "not mentioned" means we did not find it in the sources, not that it is certainly missing.

2.2 Similar Solutions

Solution	What it offers	Strengths	Limitations
Tripadvisor [1] International travel app	Reviews of hotels, restaurants, tours and attractions; nearby places; saves shown on a custom map; itineraries organized by day; AI itinerary (beta); tickets and reservations	Very large community content; saves plus map plus day itinerary in one place; photos and reviews	General travel app (hotels, restaurants, booking) rather than a focused attractions guide; not Egypt-specific; no interest-based onboarding or alerts for saved places mentioned
Wanderlog [2][3] Trip planner	Itinerary building, budget, reservations, places on a map, collaboration with friends, export to Google Maps (Pro), plans stored offline	Strong planning and sharing; Google Maps link; offline plans	Large feature set can be complex; some features need the paid plan and a review notes limited offline use on the free plan; no location alerts mentioned
VoiceMap [4][5] GPS audio tours	Audio tours that start automatically by GPS at each location; downloaded tours work offline with an offline map; directions to the start point	Shows that location-triggered content works well; works offline	Fixed paid routes, not a browsable list of attractions; the store page warns that location use in the background can reduce battery
Explore Egypt [6] Egypt guide (Android)	About 1,000 places across governorates; photos, visiting tips, opening hours, ticket prices, weather, emergency numbers; one-click directions; favorites	Rich Egypt-specific details such as hours and prices; favorites list	Source is the developer's portfolio page; personalization by interests, alerts and trips are not mentioned
Egypt Tour Guide [7] Egypt guide (iOS)	Map and attractions list sorted by distance to the map center; filter by city, category or keyword; favorites; custom markers such as the hotel	Distance sorting, filters and hotel marker are close to our ideas	Brief information about top attractions only; no accounts, alerts or trips mentioned; few ratings yet
Nile Guide [8] Student project (Al-Ahram Canadian University, 2026)	AI chatbot, text translation, image recognition and attraction information; the user picks a city, budget, number of days and interests	Confirms the value of city plus interests onboarding; AI features	Wider platform with AI and services (more complex); proximity alerts and a personal visit list are not mentioned

2.3 Comparison with Our Project

Feature	TA	WL	VM	EE	ETG	NG	Ours
Attractions browsing	Yes	Partly	Partly	Yes	Yes	Yes	Yes
Search or filter by city / category	-	-	-	Partly	Yes	Partly	Yes
Personalized by user interests	Partly	-	-	-	-	Yes	Yes
Distance / nearby places	Yes	-	Partly	-	Yes	-	Yes
Directions via maps app	-	Partly	Yes	Yes	-	-	Yes
Location-triggered alert or audio	-	-	Yes	-	-	-	Yes
Saved list / favorites	Yes	Yes	-	Yes	Yes	-	Yes
Trips or day itinerary	Yes	Yes	-	-	-	Partly	Yes
Offline use	-	Partly	Yes	-	-	-	Bonus
Ratings and reviews	Yes	-	-	-	-	-	Bonus
Egypt focus	-	-	-	Yes	Yes	Yes	Yes

TA = Tripadvisor, WL = Wanderlog, VM = VoiceMap, EE = Explore Egypt, ETG = Egypt Tour Guide, NG = Nile Guide. Yes = described in the sources; Partly = limited; "-" = not mentioned in the sources we reviewed.

2.4 What We Learn and How We Differ

Ideas we reuse

- From Tripadvisor and Wanderlog: saved places that can be grouped into trips and organized by day (our visit list, trips and day planner).
- From Wanderlog and VoiceMap: offline use is valuable for travelers (our offline mode bonus).
- From VoiceMap: location-based triggers work, but background location costs battery, so we read location only every few minutes and explain the permission clearly.
- From Explore Egypt: opening hours, ticket prices and practical tips make a details page useful (we show photos, description, opening hours and rating).
- From Egypt Tour Guide: a hotel marker and distance sorting help visitors; our onboarding asks for the hotel and sorts nearest-first.
- From Nile Guide: choosing the city and interests first makes recommendations relevant (our onboarding and featured places).

What makes our project different

- Combines in one free, Egypt-focused app: interests-based featured places, nearest-first distance, Google Maps directions, **alerts when the user is close to a place they saved**, and a visit list with trips.
- Focused scope (no booking or AI), so the team can finish a complete, tested app in the available time.

Limits compared with them: less content (depends on the places service), no booking or reviewer community, and alerts and distance need location permission.

2.5 References

[1] Tripadvisor: Plan & Book Trips, Apple App Store listing. https://apple.co/2ML2tfI

[2] Wanderlog - Travel Planner, Apple App Store listing. https://apps.apple.com/IL/app/id1476732439

[3] Wanderlog review and pros/cons, InsideA. https://insidea.com/spotlight/listing/wanderlog/

[4] VoiceMap: Audio Tours & Guides, Apple App Store listing. https://apps.apple.com/app/voicemap/id852027939

[5] VoiceMap, Walking Tour App. https://voicemap.me/walking-tour-app

[6] Explore Egypt, developer project page. https://workingnotworking.com/projects/414829-explore-egypt

[7] Egypt Tour Guide, Apple App Store listing. https://apps.apple.com/us/app/-/id6474905513

[8] Al-Fanar Media, "Student project puts Egypt's tourist services into one app", Aug 2026. https://al-fanarmedia.org/2026/08/student-project-puts-egypts-tourist-services-into-one-app/

All sources accessed on 19 Oct 2026. Features of third-party apps can change; verify them again before the final presentation.

Part 3

Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Type	Needs and expectations	How the project responds
Tourists visiting Egypt	Primary user	Find interesting places fast, know the distance, get directions, keep a list of places, be reminded when close	Featured places, distance, Google Maps button, visit list, nearby alerts
Egyptians visiting another city	Primary user	Discover places that match their interests in an unfamiliar city	Residence, visited city and interests in onboarding; filters and search
Development team (5 members)	Internal	Clear scope, equal workload, tasks that match the time available	Task split in Section 1.3, risk plan, rotation of testing
Technical instructor and DEPI evaluators	Reviewer	Complete documentation on time, a working app, GitHub history, a clear demo	Documentation plan, milestones on DEPI deadlines, tests and reports
Service providers (places API, Google Maps, Firebase)	External	Usage within terms and quotas; API keys kept private	Free plans, caching, keys outside the repository
Tourism sector and local attractions	Indirect	Correct information and more visitors	Details page with hours and rating; data taken from the places service

3.2 User Flow

First time only	Every time the user opens the app	On the trip
1. Log in or sign up (email or Google) 2. Choose residence (home country and city) 3. Choose where you are visiting (GPS or list) 4. Where are you staying? (hotel name and location, can change anytime) 5. What do you like? (history, nature, beaches, food, shopping)	6. See featured places that match your likes, closest first 7. Explore more: search, filter (Historical / Recreational), see the distance 8. Open place details (photos, description, opening hours, rating) 9. Save to your visit list and group places into trips	10. Navigate: one tap opens directions in Google Maps 11. Nearby alert when close to a saved place 12. Mark as visited, add notes (rating and badges are bonus features)

3.3 User Stories

ID	User story	Acceptance criteria
US1	As a visitor, I want to sign up with my email or Google account so that my data is saved.	Account created in under 1 minute; I stay logged in after closing the app
US2	As a user, I want to set my residence, the city I visit and my hotel so that the app knows where I am based.	City can be chosen by GPS or from a list; hotel can be edited later
US3	As a user, I want to choose what I like so that I see relevant places.	At least one interest can be selected; featured places match the interests
US4	As a tourist, I want a list of historical and recreational attractions with search and filters so that I can find places quickly.	List loads for the chosen city; search by name and filter by category work
US5	As a tourist, I want a details page so that I can decide if a place is worth visiting.	Photos, description, opening hours and rating are shown (missing data shows a default)
US6	As a tourist, I want the nearest places first with the distance so that I save time.	Each place shows distance; the list is sorted nearest first
US7	As a tourist, I want to open directions in Google Maps so that I can get there easily.	One tap opens Google Maps with the place as destination
US8	As a tourist, I want an alert when I am close to a place I saved so that I do not miss it.	One alert per saved place when I enter the radius; can be turned off in settings
US9	As a tourist, I want a visit list so that I can plan and track my visits.	I can add, remove, mark visited and write notes; places can be grouped into trips
US10	As a returning user, I want my ratings to improve my featured places (bonus).	Rated places influence the featured list next time

3.4 Use Cases

ID	Use case	Actor	Main flow	Alternative / exception
UC1	Sign up / log in	User	Enter email or choose Google; account is created or opened; app shows onboarding or home	Wrong password: error and retry; no internet: message
UC2	Set residence, visited city, hotel and interests	User	Choose home country and city; choose visited city by GPS or list; enter hotel; select interests; save	GPS off: choose from list; skip hotel and add it later
UC3	Browse, search and filter attractions	User, places service	App loads places for the city; user searches by name or filters Historical / Recreational	Service unavailable: show saved places and a message
UC4	View place details	User	User opens a place; app shows photos, description, opening hours and rating	Data missing: show default text
UC5	See nearest places with distance	User	App reads location, calculates distance to each place and sorts nearest first	Permission denied: use hotel or city center as the start point
UC6	Open directions in Google Maps	User, Google Maps	User taps Navigate; Google Maps opens with the destination	Maps app missing: open in the browser
UC7	Receive nearby alert	User, system	App checks location every few minutes; when within the radius of a saved place it shows a notification	Alerts off or permission denied: no alert; place already notified: no repeat
UC8	Manage visit list and trips	User	User adds or removes a place, marks it visited, writes notes, groups places into a trip	Place already in list: show message
UC9	Change settings	User	User turns alerts on or off and changes language	-

3.5 Functional Requirements

Priority: **Must** = the 9 must-have features; **Bonus** = one per member (2 or 3 are enough if time is short).

ID	Requirement	Priority	Owner
FR1	The user can sign up and log in with email and password	Must	M1
FR2	The user can sign in with a Google account	Must	M1
FR3	The user can choose a residence (home country and city) by list	Must	M1
FR4	The user can choose the city they are visiting by GPS or from a list	Must	M1
FR5	The user can enter and edit the hotel name and location	Must	M1
FR6	The user can choose interests: history, nature, beaches, food, shopping	Must	M1
FR7	The user can edit settings: alerts on/off and language	Must	M1
FR8	The system gets attractions of the chosen city from an external places service and stores them in the database	Must	M2
FR9	The system lists attractions categorized as Historical or Recreational	Must	M2
FR10	The user can search attractions by name and filter by category	Must	M2
FR11	The system shows featured places that match the user's interests	Must	M2
FR12	The details page shows photos, description, opening hours and rating	Must	M2
FR13	The system calculates the distance from the user (or hotel) to each place	Must	M3
FR14	The system sorts places nearest first and shows the distance	Must	M3
FR15	'Open in Google Maps' and 'Navigate' buttons open directions to the place	Must	M3
FR16	The app asks for location and notification permission and explains why	Must	M4
FR17	The app reads the user's location periodically (every few minutes) while permission is granted	Must	M4
FR18	The app shows an alert when the user is within a set radius of a saved place (proposed default 300 m)	Must	M4
FR19	The same place does not trigger repeated alerts within a cooldown period; alerts respect the settings	Must	M4
FR20	The user can add a place to and remove a place from the visit list	Must	M5
FR21	The user can mark a place as visited and write notes	Must	M5
FR22	The user can group places into trips	Must	M5
B1	Ratings and reviews that improve featured places	Bonus	M1
B2	Offline mode with saved places	Bonus	M2
B3	Share a trip with a friend	Bonus	M3
B4	Badges, for example "Visited 10 places"	Bonus	M4
B5	Day planner that orders places nearest first	Bonus	M5

3.6 Non-Functional Requirements

Category	ID	Requirement
Performance	NFR1	The attraction list loads in under 3 seconds on mobile data (under 1 second from saved data); the app starts in under 3 seconds
Performance	NFR2	Distance calculation and sorting for 100 places take under 1 second
Reliability	NFR3	Crash-free sessions of 99% or higher; if the internet or places service fails, the app shows saved data and a clear message
Reliability	NFR4	A nearby alert appears within 5 minutes of entering the radius; fewer than 5% of test visits give a duplicate or false alert
Security	NFR5	Login is handled by Firebase Authentication; the app never stores passwords itself
Security	NFR6	Firestore security rules allow each user to read and write only their own data; all network traffic uses HTTPS
Security	NFR7	API keys and secrets are not committed to GitHub
Privacy	NFR8	Location is used only for distance and alerts; the user is told why it is needed and can turn alerts off; only the minimum profile data is stored
Usability	NFR9	First-time setup (login to featured places) takes under 2 minutes; the SUS score is 70 or higher
Usability	NFR10	Consistent design from the Figma screens, readable text and contrast, buttons large enough to tap, English UI with Arabic as a language option in settings if time allows
Efficiency	NFR11	Background location checks use less than 5% battery per hour
Compatibility	NFR12	Runs on current Android phones (and iOS if a device is available) and on different screen sizes
Maintainability	NFR13	Clean, commented code with naming conventions; one feature per folder; tests for the main logic (60% coverage target); changes through reviewed pull requests

3.7 Constraints and Assumptions

- Final deadlines: documentation 19 Oct 2026, design 6 Nov, implementation 30 Nov, final delivery 4 Dec; the defense date will be set by DEPI.
- The team has 5 members who also study other courses, so the scope is the 9 must-have features plus a few bonus features.
- Attraction data comes from an external places service chosen by M2; its free plan and data quality limit what we can show.
- Alert radius (300 m) and the check interval are proposed values and will be tuned during testing on real phones.